

第4章: JavaScript / TypeScript — 画面のロジックを書く言語（完全版）

前の章（03）では、画面の **骨組み（HTML）** と **見た目（CSS）** を学びました。でも、それだけでは画面は「動き」ません。「ボタンを押したらログインする」「入力した値をサーバーに送る」「結果に応じてエラーを表示する」——こうした **動き（ロジック）** を担当するのが **JavaScript** と、その進化版である **TypeScript** です。

この章では、まずJavaScript/TypeScriptの文法を **一通り網羅** し、そのうえで本アプリの実ファイル `Login.tsx` ・ `KosuList.tsx` ・ `axios.ts` を **1行残らず** 読み解きます。とくに「ログインボタンを押したときに何が起きるか（`handleSubmit`）」と「サーバーと通信する非同期処理（`async / await`）」を、たとえ話を交えて徹底的に解説します。

この章は長いですが、**辞書のように使えること** を目指しています。実務で「`map` ってどう書くんだっけ」「`async` の意味は？」と思ったら、ここに戻って該当箇所を引いてください。

この章で学ぶこと

- **JavaScriptとは何か** — ブラウザを動かす言語。Reactの土台
- **基本文法を網羅** — コメント・変数（`const / let`）・データ型・演算子（`===`）・テンプレートリテラル・`if` ・ `for` ・ 関数・アロー関数
- **配列とオブジェクト** — `map` ・ `filter`、オブジェクト、分割代入、スプレッド、オプションチェーン（`?.`）
- **TypeScriptの型** — 型注釈・型推論・`interface` ・ 型エイリアス・配列型・union型。「なぜ型が必要か」
- **非同期処理** — `Promise` ・ `async / await` を、レストランの注文票でたとえて完全理解
- **import / export** — ファイルをまたいでコードを使い回すしくみ
- **本アプリの `axios.ts` 全8行を1行ずつ** — サーバーとの通信窓口の作り方
- **本アプリの `Login.tsx` の `handleSubmit` を1行ずつ** — ログイン処理の全貌
- **本アプリの `KosuList.tsx` の `interface Kosu` ・ `formatTyoku` ・ `getDayOfWeek` ・ `map / filter` を1行ずつ**
- **トラブルシューティング・演習問題**

目次

- 0. 前提知識：JavaScriptとTypeScriptって何？
- 1. コメントの書き方
- 2. 変数 — 値に名前を付ける（const と let）
- 3. データ型 — 値の種類
- 4. 演算子 — 計算と比較（=== の重要性）
- 5. テンプレートリテラル — 文字列に値を埋め込む
- 6. 条件分岐 — if / else / 三項演算子 / switch
- 7. 繰り返し — for とループ
- 8. 関数 — 処理に名前を付けて再利用
- 9. アロー関数 — 関数の短い書き方
- 10. 配列 — 値を並べて持つ
- 11. 配列の必殺技 — map と filter
- 12. オブジェクト — 名前付きの値の集まり
- 13. 分割代入・スプレッド・オプショナルチェイニング
- 14. TypeScript — 型という安全装置
- 15. 非同期処理 — Promise と async / await
- 16. import / export — ファイルをまたぐ
- 17. 実コード全解説：axios.ts 全8行
- 18. 実コード全解説：Login.tsx の handleSubmit
- 19. 実コード全解説：KosuList.tsx の型と関数とmap
- 20. トラブルシューティング
- 21. 演習問題
- 22. この章のまとめ

0. 前提知識：JavaScriptとTypeScriptって何？

0.1 JavaScriptの立ち位置

03 章で、画面は3つの言語できていると学びました。もう一度おさらいします。

種類	役割	家でたとえると
----	----	---------

HTML	構造・内容	骨組み（柱・壁）
CSS	見た目	内装・塗装
JavaScript	動き・ロジック	電気・設備（スイッチを押すと電気がつく）

HTMLとCSSだけだと、画面は「絵」のように静止しています。ボタンを描くことはできても、「押したら何かが起きる」ようにはできません。その「押したら～する」を実現するのが **JavaScript** です。

JavaScript（ジャバスクリプト：Webブラウザの中で動かすために生まれたプログラミング言語）とは？ もともと「Webブラウザの中で動かすための言語」として生まれたプログラミング言語。略して **JS** とも書きます。今ではブラウザだけでなく、サーバー（Node.js）など色々な場所で動きますが、本アプリでは主に「ブラウザの中で画面を動かす」ために使われます。⚠️ 名前が似ていますが「**Java**」と「**JavaScript**」はまったくの別物です。「インド」と「インドネシア」くらい違います。混同しないように。

0.2 TypeScriptは「JavaScript＋型」

本アプリのファイルは `.tsx` や `.ts` という拡張子です。これは **TypeScript** のファイルです。

TypeScript（タイプスクリプト：JavaScriptに「型」を足した言語）とは？ JavaScriptに「**型（かた）**」というしくみを足した言語。略して **TS**。Microsoftが作りました。JavaScriptにできることは全部でき、さらに「この値は数値」「この値は文字列」というラベル（型）を付けられます。これにより、書いている途中で間違いに気づけます（§14で詳しく）。

TypeScriptは、最終的に **JavaScriptに変換（コンパイル）** されてからブラウザで動きます。ブラウザはTypeScriptを直接は理解できないからです。この変換は、本アプリでは自動で行われるので、あなたが意識する必要はありません。

`.ts` と `.tsx` の違い:

- `.ts` … 普通のTypeScript（ロジックだけ）。本アプリでは `axios.ts` がこれ。
- `.tsx` … TypeScriptの中に **JSX**（HTMLのような画面記述、03 §3参照）を書けるファイル。Reactの画面ファイルはこちら。 `Login.tsx` ・ `KosuList.tsx` がこれ。

0.3 この章の読み方

§1～§16は **文法の教科書** です。説明用の小さな例（「※説明用の簡易例」と明記）で文法を学びます。§17～§19は **本アプリの実コード** を1行ずつ読みます。文法の知識を、本物のコードで確かめる段階です。

最初は文法を「ふんふん」と眺め、§17以降の実コードで「あ、これさっきの文法だ」と気づければ大成功です。

1. コメントの書き方

コードの中に書く「メモ書き」です。コンピュータは無視し、人間が読むためのものです。

▼ **このコードがやること（先に日本語で）**：プログラムの動きには一切影響しない「人間向けの説明文」を書く方法を示します。

```
// これは1行コメント。// から行末までが無視される（※説明用の簡易例）

const x = 5; // 行の途中からコメントを始めることもできる

/*
  これは複数行コメント。
  この範囲がすべて無視される。
  長い説明を書きたいときに使う。
*/
```

- `//` … その行の、`//` から右側だけがコメント（**1行コメント**）。最頻出。
- `/* ... */` … 囲んだ範囲すべてがコメント（**複数行コメント**）。

なぜ？ コードは「何をするか（What）」は読めば分かりますが、「なぜそうするか（Why）」は書いた人にしか分かりません。半年後の自分や後任者のために、「なぜこの処理が必要か」をコメントで残すのがプロの習慣です。

⚠ JSX（HTMLのような部分）の中では `//` は使えず、`{/* ... */}` と書きます（03 §3で見た形）。場所によって書き方が変わるので注意。

2. 変数 — 値に名前を付ける（`const` と `let`）

2.1 変数とは

用語: 変数（へんすう：値を入れておく名前付きの箱） とは、値を入れておく「名前付きの箱」です。

たとえ話: 「ロッカー」を想像してください。ロッカーに番号（名前）を付け、中に物（値）を入れます。後で「3番ロッカーの中身」と言えば取り出せます。変数も同じで、「`employee_no` という名前の箱に `123` を入れる」→ 後で `employee_no` と書けば `123` が取り出せます。

▼ **このコードがやること（先に日本語で）:** `employeeName` という箱に文字列「山田」を入れ、`age` という箱に数値42を入れます。

```
const employeeName = "山田"; // employeeName という箱に "山田" を入れる（※説明用の簡易例）
let age = 42;                // age という箱に 42 を入れる
```

分解すると：

- `const` または `let` … 「これから箱を作るよ」という宣言キーワード。
- `employeeName` … 箱の名前（**変数名**）。自分で好きに付けられます。
- `=` … 「右の値を左の箱に入れる」という代入記号（**等しい、ではない！** §4参照）。
- `"山田"` … 箱に入れる値。

2.2 `const` と `let` の違い（超重要）

JavaScriptで変数を作る方法は主に2つ。**この使い分けは毎日使う基本**です。

キーワード	意味	後で中身を変えられる？
<code>const</code>	定数 (constant)。一度入れたら変えない箱	❌ 変えられない
<code>let</code>	変数。後で中身を入れ替えられる箱	✅ 変えられる

▼ このコードがやること: `const` の箱は中身を変えようとするエラー、`let` の箱は変えられることを示します。

```
const pi = 3.14; // 円周率。変わらない値なので const (※説明用の簡易例)
pi = 3.15;      // ❌ エラー! const で作った箱の中身は変えられない

let count = 0;  // カウンター。後で増やすので let
count = 1;      // ✅ OK。let なら中身を入れ替えられる
count = count + 1; // ✅ OK。今の count(1) に 1 を足して count に入れ直す → 2
```

なぜ `const` を優先するの? 「変えるつもりのない箱」を `const` にしておくのと、うっかり書き換えてしまうミスは **コンピュータが止めてくれます**。プロは「まず `const` で書き、どうしても変える必要があるときだけ `let`」という習慣です。本アプリのコードを見ても、`const` が圧倒的多数です。

⚠️ 昔は `var` (バー) というキーワードもありましたが、紛らわしい挙動があるため、現代では **使いません**。本アプリにも出てきません。`const` と `let` だけ覚えればOKです。

2.3 変数名のルール

- 英字・数字・`_`・`$` が使える。ただし **数字で始めてはいけない** (`1name` はダメ、`name1` は OK)。
- 大文字と小文字は区別される (`age` と `Age` は別の箱)。
- 意味が分かる名前を付ける (`x` より `employeeName` のほうが親切)。
- 単語をつなぐときは **キャメルケース** (`employeeName` のように2語目以降の頭を大文字) が慣例。

用語: キャメルケース（camelCase：単語の切れ目を大文字で表す命名法） `employeeName`、`searchDayRef` のように、ラクダ（camel）のコブのように大文字が出っ張る様子から。本アプリの変数名はほぼこれです。

3. データ型 — 値の種類

変数に入れる「値」には、いくつかの **種類（型）** があります。代表的なものを押さえます。

3.1 基本の型（プリミティブ型）

▼ **このコードがやること:** 文字列・数値・真偽値・null・undefined という、JavaScriptの基本的な値の種類を一通り作ってみます。

```
const name = "山田太郎";    // 文字列（string）。"" か '' で囲む（※説明用の簡易例）
const age = 42;              // 数値（number）。整数も小数も同じ number
const isAdmin = true;        // 真偽値（boolean）。true（真）か false（偽）の2択
const nothing = null;        // null（ヌル）。「意図的に空」を表す
let notYet;                  // undefined（アンディファインド）。「まだ値を入れていない」
```

型	英語名	意味	例
文字列	string	文字の並び。引用符で囲む	"山田", 'abc'
数値	number	数。整数も小数も区別なし	42, 3.14, -5
真偽値	boolean	「はい/いいえ」の2択	true, false
null	null	「意図的に空っぽ」	null
undefined	undefined	「まだ何も入っていない」	undefined

用語: 文字列（string／ストリング：文字の並び） 必ず引用符で囲みます。"123" は **文字列の123**（計算できない）で、123 は **数値の123**（計算できる）です。この違いは後でとても重要になります（§18のログイン処理で `Number()` が出てきます）。

用語: 真偽値 (boolean／ブーリアン：trueかfalseのどちらかしか取らない型) 「ログイン済みか?」「管理者か?」のような Yes/No を表すのに使います。本アプリの `interface Kosu` の `judgement: boolean` (整合性OK/NG) がこれです (§ 19)。

なぜ? null と undefined の違い: どちらも「値がない」ですが、ニュアンスが違います。

- `null` … 「ここは空ですよ」と **人間が意図的に** 入れた空。
- `undefined` … 「箱は作ったが、まだ何も入れていない」という **自然な空っぽ**。実務では「データがまだ来ていない = `null`」のように使い分けます。本アプリの `useState<string | null>(null)` (§ 19) は「最初はエラーなし = null」という使い方です。

3.2 型を調べる typeof

ある値が何型かを調べるには `typeof` を使います。

```
typeof "山田" // "string" (※説明用の簡易例)
typeof 42     // "number"
typeof true  // "boolean"
```

4. 演算子 — 計算と比較 (=== の重要性)

用語: 演算子 (えんざんし：operator) とは、`+` や `===` のような「計算・比較の記号」です。

4.1 算術演算子（計算）

```
const a = 10 + 3; // 13（足し算）（※説明用の簡易例）
const b = 10 - 3; // 7（引き算）
const c = 10 * 3; // 30（掛け算。×ではなく*）
const d = 10 / 3; // 3.333...（割り算。÷ではなく/）
const e = 10 % 3; // 1（余り。10を3で割った余り。「剰余」という）
```

用語: %（剰余演算子／じょうよ：割り算の余りを求める記号） 「偶数か奇数か」（`x % 2` が 0なら偶数）の判定などに使います。03章のCSSにも `nth-child(even)` がありましたが、JSでも % で偶奇判定をよくします。

4.2 文字列の連結

+ は、文字列どうしだと「つなげる」意味になります。

```
const greeting = "こんにちは、" + "山田さん"; // "こんにちは、山田さん"（※説明用の簡易例）
const message = "番号は" + 123 + "です"; // "番号は123です"（数値が文字列に変換される）
```

⚠ + は「数値どうしなら足し算、文字列が混じると連結」という二面性があります。 `"1" + 2` は `** 3` ではなく `"12" **`（文字列の連結）になります。これがバグの温床になることがあるので、§5のテンプレートリテラルのほうが安全です。

4.3 比較演算子（== と === の違い）★最重要★

2つの値を比べて、`true` か `false` を返します。

```
5 > 3 // true（5は3より大きい）（※説明用の簡易例）
5 < 3 // false
5 >= 5 // true（以上）
5 <= 4 // false（以下）
5 === 5 // true（厳密に等しい）
5 !== 3 // true（厳密に等しくない）
```

★最重要★ `===`（イコール3つ）を使うこと: JavaScriptには「等しい」の比較が2種類あります。

- `==`（イコール2つ）… **ゆるい比較**。型が違ってても「だいたい同じ」なら `true` にしてしまう。`"5" == 5` は `true`（文字列の5と数値の5を同じ扱い）。
- `===`（イコール3つ）… **厳密な比較**。型まで一致しないと `true` にしない。`"5" === 5` は `false`（文字列と数値は別物）。

`==` は予想外の挙動でバグを生むため、現代のJavaScriptでは **必ず `===` を使う** のが鉄則です。本アプリのコードも `===` だけを使っています。 `Login.tsx` の `data.status === "success"` (§18)、 `KosulList.tsx` の `value === ""`、 `err.response?.status === 401` (§19) など、すべて `===` です。

⚠ 代入の `=` と比較の `===` を混同しない:

- `=`（1つ）… 代入（箱に入れる）。`x = 5` は「xに5を入れる」。
- `===`（3つ）… 比較（等しいか調べる）。`x === 5` は「xは5と等しいか?」。初心者は `if (x = 5)` と書いてしまいがちですが、これは「比較」のつもりが「代入」になってバグります。比較は必ず `===`。

4.4 論理演算子（かつ・または・でない）

複数の条件を組み合わせます。

```
true && true    // true (&& は「かつ (AND)」。両方trueならtrue) (※説明用の簡易例)
true && false   // false
true || false  // true (|| は「または (OR)」。どちらかtrueならtrue)
!true         // false (! は「でない (NOT)」。反転させる)
```

記号	名前	意味	覚え方
<code>&&</code>	AND	両方ともtrueのときtrue	「かつ」
<code> </code>	OR	どちらかがtrueならtrue	「または」
<code>!</code>	NOT	true/falseを反転	「～でない」

&& の応用（本アプリで多用）：`{errorMessage && (<div>...</div>)}`（03 §3⑨で見た形）は、「`errorMessage` に中身があるときだけ右側を表示」という意味です。React独特の使い方ですが、根っこは **&&** です（05 章で詳しく）。

|| の応用（既定値）：`response.data.results || []`（§19の `KosuList.tsx`）は、「`results` があればそれを使い、なければ（空っぽなら）右の `[]` を使う」という「**ないときの保険**」です。本アプリで頻出のテクニックです。

5. テンプレートリテラル — 文字列に値を埋め込む

文字列の中に変数の値を埋め込む、便利な書き方です。

▼ **このコードがやること**：「山田」と42という変数の値を、文章の中に直接差し込んで1つの文字列を作ります。

```
const name = "山田"; // （※説明用の簡易例）
const age = 42;

// 古い書き方（+ で連結）→ 読みにくい
const msg1 = "私は" + name + "、" + age + "歳です";

// テンプレートリテラル（バッククォート ` で囲み、${ } で値を埋める）→ 読みやすい
const msg2 = `私は${name}、${age}歳です`;
// どちらも結果は "私は山田、42歳です"
```

分解すると：

- 文字列を **バッククォート** ```（Shift+@ などを入力する記号。シングルクォート `'` とは別物）で囲む。
- 中に **`${変数や式}`** と書くと、その部分が **値に置き換わる**。

用語: テンプレートリテラル (template literal: バッククォートで囲み `${ }` で値を埋め込む文字列) 「テンプレート (ひな形) に値を流し込む」イメージ。+ の連結より読みやすく、ミスも減ります。

本アプリでの実例: `KosuList.tsx` の `to={` /kosu-update/${item.id}`}` (§ 19) がこれです。「`/kosu-update/`」という固定文字列の後ろに、`item.id` (例: 5) をくっつけて `/kosu-update/5` というURLを作る」処理です。`${ }` の中に `item.id` という値を埋め込んでいます。

6. 条件分岐 — if / else / 三項演算子 / switch

「もし〜なら〇〇、そうでなければ××」という分かれ道を作ります。

6.1 if / else

▼ このコードがやること: 年齢が18以上なら「成人」、そうでなければ「未成年」と判定します。

```
const age = 20; // (※説明用の簡易例)

if (age >= 18) {      // もし age が18以上なら (条件は ( ) の中)
  console.log("成人"); // この { } の中を実行
} else {             // そうでなければ
  console.log("未成年"); // こちらの { } を実行
}
```

分解すると：

- `if (条件) { ... }` … 条件が `true` なら `{ }` の中を実行。
- `else { ... }` … 条件が `false` のときに実行 (省略可能)。
- 条件は丸カッコ `( )` の中、実行内容は波カッコ `{ }` の中に書く。

用語: `console.log(...)` (コンソールログ: カッコの中の値をConsoleに出力する命令) ブラウザの **開発者ツールのConsoleタブ** (03 §14.3) に値を出します。「今この変数に何が入っているか」を確認するための、デバッグ (不具合調査) の定番ツールです。

3つ以上に分岐したいときは `else if` をはさみます。

```
if (score >= 80) {           // (※説明用の簡易例)
  console.log("優");
} else if (score >= 60) { // 上がfalseで、これがtrueなら
  console.log("良");
} else {                   // どれもfalseなら
  console.log("可");
}
```

6.2 三項演算子 (短いif/else)

`if/else` を **1行で書く** 書き方です。本アプリで頻出します。

```
// 構文: 条件 ? trueのときの値 : falseのときの値
const result = age >= 18 ? "成人" : "未成年"; // (※説明用の簡易例)
// age >= 18 が true なら "成人"、false なら "未成年" が result に入る
```

- `条件 ? A : B` … 条件がtrueならA、falseならBを返す。
- `?` の前が条件、`?` と `:` の間がtrueの値、`:` の後がfalseの値。

本アプリでの実例 (3つ):

- `Login.tsx`: `setNumber(value === "" ? "" : value)` → 「入力が空文字なら空文字を、そうでなければその値を」セットする (§18)。
- `KosuList.tsx`: `{item.judgement ? "OK" : "NG"}` → 「judgementがtrueなら "OK"、falseなら "NG" を表示」 (§19)。
- `KosuList.tsx`: `mode ? "month" : "day"` → 「monthモードなら "month"、そうでなければ "day"」 (§19)。

6.3 switch（値ごとの分岐）

1つの値が「1なら～、2なら～、3なら～」と多数に分かれるとき、`if` を並べるより読みやすいのが `switch` です。

▼ このコードがやること: 数値1～6に応じて、対応する勤務シフト名の文字列を返します。本アプリの `formatTyoku` (§ 19で実コード解説) の構造そのものです。

```
switch (value) {                // value の値で分岐（※説明用の簡易例）
  case 1:                       // value が 1 のとき
    return "1直";              // "1直" を返してswitchを抜ける
  case 2:                       // value が 2 のとき
    return "2直";
  case 3:                       // value が 3 のとき
    return "3直";
  default:                     // どの case にも当てはまらないとき
    return "";                 // 空文字を返す
}
```

分解すると：

- `switch (調べる値) { ... }` … カッコの値を、下の `case` と順に照合する。
- `case 値:` … 「調べる値がこれと一致したら」。一致した case の処理が実行される。
- `return ...` … 値を返して関数を終える（`return` がない場合は `break` で抜ける必要あり。§ 19で解説）。
- `default:` … どの `case` にも一致しなかったときの受け皿（`if` の最後の `else` に相当）。

なぜ？ switch vs if: 「1つの値を、たくさんの具体的な値と照合する」なら `switch` がスッキリします。「複雑な条件（範囲やAND/OR）」なら `if` が向きます。本アプリは「直（シフト番号）→名前」の変換に `switch` を選んでいます。

7. 繰り返し — for とループ

同じ処理を何度も繰り返すしくみです。

7.1 基本の for

▼ このコードがやること: 0から4までの数字を順番に5回、コンソールに表示します。

```
for (let i = 0; i < 5; i++) { // i を0から始め、5未満の間、毎回1ずつ増やす（※説明用の簡易例）
  console.log(i);           // i を表示（0,1,2,3,4 と5回出る）
}
```

for のカッコの中は **セミコロン ;** で3つに区切られています：

1. `let i = 0` … **初期化**。ループ用のカウンター変数 `i` を0からスタート。
2. `i < 5` … **継続条件**。これがtrueの間、繰り返す。falseになったら終了。
3. `i++` … **更新**。1周ごとに `i` を1増やす（`i++` は `i = i + 1` の短縮形）。

用語: ループ (loop : 繰り返し処理) 「グルグル回る」イメージから。 `i` がカウンター（数え役）として0→1→2…と増え、`i < 5` が false（5になった瞬間）でループを抜けます。

`i++` と `i--` : `i++` は「1増やす」（インクリメント）、`i--` は「1減らす」（デクリメント）。
`+= 2` なら「2ずつ増やす」も書けます。

7.2 配列を1つずつ処理する (for...of)

配列 (§ 10) の要素を1つずつ取り出すには `for...of` が便利です。

```
const fruits = ["りんご", "みかん", "ぶどう"]; // （※説明用の簡易例）
for (const fruit of fruits) { // fruits の要素を1つずつ fruit に入れて繰り返す
  console.log(fruit);        // "りんご" → "みかん" → "ぶどう" の順に表示
}
```

なぜ？ React開発では for をあまり書きません。 配列の処理は § 11 の `map` ・ `filter` という、もっと洗練された方法を使うからです。本アプリの実コードにも `for` はほぼ出てこず、

`map` が主役です。ただし `for` の考え方（繰り返し）は基礎として大切なので押さえておきましょう。

8. 関数 — 処理に名前を付けて再利用

用語: 関数（かんすう：function。一連の処理に名前を付けて何度でも呼び出せるようにしたもの）

たとえば: 「自動販売機」を想像してください。お金（入力）を入れてボタンを押すと、ジュース（出力）が出てきます。中の仕組みを毎回気にせず、「お金を入れる→ジュースが出る」だけ使えます。関数も同じで、「値を渡す→結果が返る」を、中身を気にせず何度でも使えます。

8.1 関数の定義と呼び出し

▼ **このコードがやること:** 2つの数を受け取って足し算した結果を返す `add` という関数を作り、それを使います。

```
function add(a, b) { // add という名前の関数。a と b を受け取る（※説明用の簡易例）
  return a + b;      // a + b の結果を「返す」
}

const result = add(3, 5); // 関数を呼び出す。3 と 5 を渡すと 8 が返り、result に入る
console.log(result);      // 8
```

分解すると：

- `function 関数名(引数1, 引数2) { ... }` … 関数の定義。
- **引数（ひきすう）** … 関数に渡す値（`a`, `b`）。「材料」のイメージ。
- `return 値;` … 関数の結果を呼び出し元に **返す**。
- `add(3, 5)` … 関数の **呼び出し**。カッコの中に渡す値（**実引数**）を書く。

用語: 引数（ひきすう／argument・parameter：関数に渡す入力値） 自動販売機でいう「入れたお金」。複数あればカンマで区切ります。

用語: 戻り値（もどりち／return value：関数が返す結果） 自動販売機でいう「出てきたジュース」。`return` で指定します。`return` がない関数は `undefined`（何も返さない）になります。

8.2 引数の型は何でもよい

引数には数値だけでなく、文字列・真偽値・配列・オブジェクト、何でも渡せます。

```
function greet(name) { // 文字列を受け取る関数（※説明用の簡易例）
  return "こんにちは、" + name + "さん";
}
greet("山田"); // "こんにちは、山田さん"
```

本アプリの関数: `KosuList.tsx` の `formatTyoku(value)`（直番号→名前）、`getDayOfWeek(dateStr)`（日付→曜日）が、まさにこの「値を受け取って、変換した値を返す」関数です（§19で実コード解説）。

9. アロー関数 — 関数の短い書き方

`function` キーワードを使わず、`=>`（アロー＝矢印）で関数を書く方法です。**Reactでは圧倒的にこちらが主流**で、本アプリもほぼアロー関数です。

9.1 基本形

▼ **このコードがやること:** §8の `add` 関数を、アロー関数で書き直します。やることは同じです。

```
// function での書き方 (§8)
function add(a, b) {
  return a + b;
}

// アロー関数での書き方 (同じ意味) (※説明用の簡易例)
const add = (a, b) => {
  return a + b;
};
```

分解すると：

- `const 関数名 = (引数) => { 処理 };` … アロー関数を変数に入れる形。
- `function` の代わりに、引数のカッコの後ろに `=>` を書く。

9.2 省略形

処理が「1つの値を返すだけ」なら、`{ }` と `return` を省略できます。

```
const add = (a, b) => a + b; // { return } を省略。a + b の結果がそのまま返る (※説明用)

const double = (x) => x * 2; // 引数1つ。x を2倍にして返す
const greet = () => "hello"; // 引数なしは () と書く
```

9.3 Reactでの「その場で書く」アロー関数

Reactでは、「ボタンを押したら～する」処理を、その場でアロー関数として書くことが非常に多いです。

```
// ボタンクリック時の処理を、その場でアロー関数で渡す (※説明用の簡易例)
<button onClick={() => handleSearch(true)}>指定月</button>
```

これは `KosuList.tsx` の実コード (§19) そのものです。「`onClick` (クリックされたとき) に、`handleSearch(true)` を呼ぶアロー関数を渡す」という意味です。

なぜ `() =>` で包むの？ `onClick={handleSearch(true)}` と書くと、**ボタンを描いた瞬間に即実行**されてしまいます。`onClick={() => handleSearch(true)}` と「関数で包む」ことで、

「クリックされたら初めて実行する」という予約ができます。この違いは 05 章でも繰り返し出てきます。

本アプリのアロー関数たち（先取り）：

- `Login.tsx` : `const handleSubmit = async (event) => { ... }`（ログイン処理本体）
- `Login.tsx` : `onChange={e => { ... }}`（入力が変わったときの処理）
- `KosuList.tsx` : `const formatTyoku = (value) => { ... }`、`const getDayOfWeek = (dateStr) => { ... }`
- `KosuList.tsx` : `data.map((item) => ( ... ))`（後述のmap）すべてアロー関数です。
`=>` を見たら「これは関数だ」と即座に読めるようにしましょう。

10. 配列 — 値を並べて持つ

用語: 配列（はいれつ：array。複数の値を順番に並べて1つの箱に入れたもの）

たとえ話: 「卵のパック」。1つの箱（パック）に、卵（値）が順番に並んで入っています。「3番目の卵」と言えば取り出せます。

10.1 配列の作り方とアクセス

▼ **このコードがやること:** 曜日の文字列を7個並べた配列を作り、番号を指定して取り出します。本アプリの `getDayOfWeek` (§ 19) の `days` 配列と同じ形です。

```
const days = ["日", "月", "火", "水", "木", "金", "土"]; // 7個の文字列の配列（※説明用の例）

console.log(days[0]); // "日"（最初の要素。番号は0から始まる！）
console.log(days[1]); // "月"
console.log(days[6]); // "土"（最後の要素）
console.log(days.length); // 7（要素の個数。length は「長さ」）
```

分解すると：

- `[ ... ]` … 角カッコで囲み、カンマで区切って値を並べる。
- `配列[番号]` … 番号（**インデックス**）で要素を取り出す。
- `.length` … 配列の要素数（個数）。

★超重要★ **番号は0から始まる**: 配列の番号（インデックス）は **1ではなく0から** 始まります。最初の要素が `[0]`、2番目が `[1]`、…、最後が `[length - 1]`。プログラミング共通のルールで、初心者が最もつまづく点です。「1番目=0番」と頭を切り替えましょう。本アプリの `getDayOfWeek` では `days[date.getDay()]` としています。 `getDay()` は日曜を0・月曜を1…土曜を6で返すので、`days` 配列の番号とピタリ一致します（\$ 19で詳説）。

10.2 要素の追加・変更

```
const list = ["a", "b"]; // （※説明用の簡易例）
list.push("c");          // 末尾に "c" を追加 → ["a","b","c"]
list[0] = "X";           // 0番目を "X" に変更 → ["X","b","c"]
```

⚠ `const` で作った配列でも、**中身の追加・変更はできます**。 `const` が禁止するのは「箱ごと別物に入れ替えること」だけ。 `list = [...]`（別の配列を代入）はダメですが、`list.push()`（中身を足す）はOKです。

11. 配列の必殺技 — map と filter

配列を扱ううえで **最も重要な2つのメソッド**（関数）です。Reactでは `for` の代わりにこれらを使います。本アプリも多用します。

用語: メソッド（method: ある値に対して `.名前()` の形で呼び出す関数） `days.length` の `length` のように、その値に「くっついた機能」のことです。

11.1 map — 全要素を変換して新しい配列を作る

▼ **このコードがやること:** 数値の配列の各要素を2倍にした、新しい配列を作ります。

```
const numbers = [1, 2, 3]; // （※説明用の簡易例）
const doubled = numbers.map((n) => n * 2); // 各要素 n を n*2 に変換
console.log(doubled); // [2, 4, 6]（元の numbers は変わらない）
```

分解すると：

- `配列.map((要素) => 変換後の値)` … 配列の **すべての要素** に対して、カッコ内の関数を適用し、**変換後の値を集めた新しい配列** を返す。
- `(n) => n * 2` … 「1個の要素 `n` を受け取り、`n * 2` に変換する」アロー関数。

たとえば: 「ベルトコンベヤー」。配列の要素が1つずつコンベヤーを流れてきて、`map` の関数（加工機）を通り、加工された状態で新しい箱に入っていく。元の箱（配列）はそのまま残ります。

★**Reactでの map は最重要**★ Reactでは「データの配列を、画面の表示（行・カードなど）の配列に変換する」のに `map` を使います。本アプリの `KosuList.tsx` では、サーバーから来た工数データの配列 `data` を、表の行 `<tr>` の配列に変換しています（§19）：

```
{data.map((item) => (  
  <tr key={item.id}> ... </tr>    // 各データ item を、1つの表の行 <tr> に変換  
))}
```

「100件データがあれば、100個の `<tr>` ができて、表に100行表示される」という仕組みです。

11.2 filter — 条件に合う要素だけ残す

▼ **このコードがやること:** 数値の配列から、偶数だけを取り出した新しい配列を作ります。

```
const numbers = [1, 2, 3, 4, 5, 6];           // (※説明用の簡易例)  
const evens = numbers.filter((n) => n % 2 === 0); // 「2で割った余りが0」=偶数だけ残す  
console.log(evens);    // [2, 4, 6]
```

分解すると：

- `配列.filter((要素) => 条件)` … 条件が `true` になる要素 **だけを集めた** 新しい配列を返す。
- `(n) => n % 2 === 0` … 「nを2で割った余りが0 (=偶数) か？」という条件。trueの要素だけ残る。

たとえば: 「ふるい (茶こし)」。配列の要素をふるいにかけて、条件に合うもの (網を通るもの) だけが下の箱に残る。

なぜ？ map と filter の違い:

- `map` … 要素の **個数は変えず**、各要素を **変換** する (3個→3個)。
- `filter` … 要素を **変換せず**、条件に合うものだけ **絞り込む** (6個→偶数の3個)。

本アプリでは、データ取得後に `|| []` (§4.4) で「結果がなければ空配列」を保証してから `map` で表示に変換する、という流れが基本です。明示的な `filter` は本ファイルでは使って

いませんが、APIへの検索条件（`filter: "true"` パラメータ、§ 19）として「絞り込み」の概念が登場します。

11.3 map と filter はつながられる

```
const numbers = [1, 2, 3, 4, 5, 6]; // （※説明用の簡易例）
const result = numbers
  .filter((n) => n % 2 === 0) // まず偶数だけ残す → [2,4,6]
  .map((n) => n * 10);       // それを10倍 → [20,40,60]
```

・ でつなげて「絞り込んでから変換」のように **数珠つなぎ** にできます（メソッドチェーンと呼びます）。

12. オブジェクト — 名前付きの値の集まり

用語: オブジェクト（object：名前（キー）と値のペアをまとめたもの） 配列が「番号で管理」なのに対し、オブジェクトは「名前で管理」します。

たとえば: 「名刺」。1枚の名刺に「名前：山田」「会社：〇〇」「電話：…」と、**項目名と値のペア** が並んでいます。オブジェクトはまさにこれです。

12.1 オブジェクトの作り方とアクセス

▼ **このコードがやること:** 1人の従業員の情報（番号・名前・管理者かどうか）を、1つのオブジェクトにまとめます。

```
const employee = {           // { } でオブジェクトを作る（※説明用の簡易例）
  employeeNo: 123,           // キー：値 のペア（カンマ区切り）
  name: "山田太郎",
  isAdmin: false,
};

console.log(employee.name);    // "山田太郎"（ドットでアクセス）
console.log(employee["name"]); // "山田太郎"（角カッコでもアクセスできる）
console.log(employee.employeeNo); // 123
```

分解すると：

- `{ キー：値, キー：値 }` … 波カッコの中に「キー（項目名）：値」をカンマ区切りで並べる。
- `オブジェクト.キー` … **ドット記法**でアクセス（最頻出）。
- `オブジェクト["キー"]` … **角カッコ記法**でもアクセス可（キーが変数のときなどに使う）。

用語: キー（key／プロパティ名：オブジェクトの各項目の名前） `employeeNo`、`name` など。
「プロパティ」とも呼びます。

本アプリでのオブジェクト（重要）：サーバーとやり取りするデータは、ほぼすべてオブジェクトです。

- `Login.tsx`：`api.post("/api/login/", { employee_no: Number(employee_no) })` → `{ employee_no: 123 }` というオブジェクトをサーバーに送る（§18）。
- `KosuList.tsx`：各工数データ `item` は `{ id, employee_no3, name, work_day2, tyoku2, judgement }` というオブジェクト。`item.id`、`item.work_day2` のようにドットで取り出します（§19）。

12.2 ネスト（入れ子）したオブジェクト

オブジェクトの値が、さらにオブジェクトになっていることもあります。


```
const response = {                                // （※説明用の簡易例）
  status: 200,
  data: {                                          // data の値が、また別のオブジェクト
    message: "成功",
    results: [1, 2, 3],
  },
};

console.log(response.data.message); // "成功"（ドットをつなげて深く入る）
console.log(response.data.results[0]); // 1（オブジェクト→配列→番号と辿る）
```

本アプリでのネスト: axiosの通信結果は `response.data.results`、
`err.response?.data.message` のように、何段もドットでつないでアクセスします（§18・
§19）。「オブジェクトの中のオブジェクトの中の値」を辿っているのです。

13. 分割代入・スプレッド・オプショナルチェイニング

ここはモダンJavaScriptの便利機能3点セット。本アプリでも使われています。

13.1 分割代入（必要な値を取り出す）

オブジェクトや配列から、必要な値を一度に取り出して変数にする書き方です。

▼ **このコードがやること:** オブジェクトから `name` と `age` を、それぞれ同名の変数に一気に
取り出します。

```
const employee = { name: "山田", age: 42, isAdmin: false }; // (※説明用の簡易例)

// 従来の書き方
const name1 = employee.name;
const age1 = employee.age;

// 分割代入 (一気に取り出す)
const { name, age } = employee; // employee.name → name、employee.age → age
console.log(name); // "山田"
console.log(age); // 42
```

- `const { キー名, キー名 } = オブジェクト;` … 同じ名前の変数に、対応する値が入る。

配列でも使えます (こちらは順番で取り出す) :

```
const [first, second] = ["a", "b", "c"]; // first="a", second="b" (※説明用の簡易例)
```

本アプリでの実例: `Login.tsx` の `const [employee_no, setNumber] = useState<string>("")` (§18) は、**配列の分割代入** です。 `useState` が返す配列の0番目を `employee_no`、1番目を `setNumber` という名前で受け取っています。Reactの `useState` はこの形で必ず使います (§05 章で詳説)。

13.2 スプレッド構文 (中身を展開する)

`...` (ドット3つ) で、配列やオブジェクトの **中身を展開** します。

▼ **このコードがやること:** 既存のオブジェクトの中身を新しいオブジェクトにコピーしつつ、項目を足します。

```
const base = { page: 1, size: 20 }; // (※説明用の簡易例)
const extended = { ...base, filter: true }; // base の中身を展開し、filter を追加
// extended は { page: 1, size: 20, filter: true }

const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4]; // arr1 を展開して [1, 2, 3, 4]
```

- `{ ...オブジェクト, 追加キー: 値 }` … 既存の中身を広げて、新しいものを足す。
- 元を壊さず「コピー＋追加」できるので、Reactの状態更新で重宝します（05章）。

本アプリでの実例: `KosuList.tsx` の `fetchData` の中：

```
params: {
  page: page,
  ...(day && {           // day に値があるときだけ、この中身を展開して足す
    day: day,
    mode: mode ? "month" : "day",
    filter: "true",
  })
}
```

これは「`page` は必ず送る。さらに `day`（日付）が入力されていれば、`day`・`mode`・`filter` も足して送る」という、**スプレッドと `&&` を組み合わせた条件付きの組み立て** です（§19で詳説）。

13.3 オプショナルチェイニング（安全にネストを辿る `?.`）

`?.`（クエスチョン＋ドット）は、「途中が `null` / `undefined` でもエラーにならずに、安全にネストを辿る」記号です。

▼ **このコードがやること:** 深いオブジェクトの値を辿るとき、途中が空っぽでもプログラムが落ちないようにします。

```
const res = { data: { message: "ok" } }; // （※説明用の簡易例）

// 普通のドット → 途中が undefined だとエラーで落ちる
console.log(res.response.data); // ✗ res.response が無いのでエラー！

// オプショナルチェイニング → 途中が無ければ undefined を返すだけ（落ちない）
console.log(res.response?.data); // undefined（エラーにならない）
```

- `a?.b` … `a` が `null` / `undefined` なら、`b` を辿らず `undefined` を返す。
- これがないと、「途中のデータが無いだけで画面全体がクラッシュ」してしまいます。

★本アプリで超頻出★ エラー処理で `err.response?.status`、`err.response?.data.message` のように使います (§ 18・§ 19)。通信エラー時、`err.response` が存在しないこと（ネットワーク断など）があるため、`?.` で安全に辿っています。「`?.` を見たら『ここは空っぽかもしれない、用心している』のサインだ」と読めればOKです。

14. TypeScript — 型という安全装置

ここからが、ただのJavaScriptに **TypeScript** が足した部分です。

14.1 なぜ型が必要か

JavaScriptは「箱に何を入れてもいい」自由な言語です。でもその自由さが、バグを生みます。

たとえ話: ラベルのない箱だらけの倉庫を想像してください。「砂糖」と思って取り出したら塩だった——こういう事故が起きます。**型** は、箱に「これは数値専用」「これは文字列専用」とラベルを貼ること。間違ったものを入れようとすると、**コンピュータがその場で警告** してくれます。

```
// JavaScript（型なし）の悲劇（※説明用の簡易例）
let age = 42;
age = "四十二";           // エラーにならない（実行するまで気づかない）
const result = age * 2;    // "四十二" * 2 → NaN（数値じゃない!）というバグ
```

```
// TypeScript（型あり）なら（※説明用の簡易例）
let age: number = 42;      // age は数値専用、と宣言
age = "四十二";           // ✖ 書いた瞬間にエディタが赤線で警告！
```

用語: NaN（ナン／Not a Number：「数値ではない」を表す特殊な値） 数値計算に文字列が混じったときなどにします。出たら「型の取り違え」を疑います。

14.2 型注釈（型を明示する）

変数名のあとに **: 型名** を書いて、型を指定します。

▼ **このコードがやること:** それぞれの変数に「この型しか入れられない」とラベルを貼ります。

```
const name: string = "山田";    // string（文字列）専用（※説明用の簡易例）
const age: number = 42;         // number（数値）専用
const isAdmin: boolean = false; // boolean（真偽値）専用
```

- **変数名: 型名 = 値;** ... **:** の後ろに型名を書く。
- 主な型名: **string**（文字列）、**number**（数値）、**boolean**（真偽値）。

14.3 型推論（書かなくても分かってくれる）

実は、初期値があれば **型注釈を省略** できます。TypeScriptが値から型を **推測（推論）** してくれるからです。

```
const name = "山田";    // 型注釈なしでも、TypeScriptが「これは string」と推論（※説明用の
name = 42;              // ✖ それでもエラー！ stringと推論されているので数値は入れられな
```

本アプリの方針: 自明なものは型推論に任せ、関数の引数・戻り値や **useState** など「分かりにくい所」だけ明示する、というバランスです。**KosuList.tsx** の **const days = ["日", ...]** には型注釈がありません（文字列配列だと推論される）。一方、関数 **formatTyoku = (value: string | number): string => ...** には引数と戻り値の型を明示しています (§19)。

14.4 関数の型注釈

関数では「引数の型」と「戻り値の型」を指定できます。

▼ **このコードがやること:** 「数値を2つ受け取り、数値を返す」関数だと明示します。

```
const add = (a: number, b: number): number => { // 引数a,bはnumber、戻り値もnumber（※説明用）
  return a + b;
};
```

- `(a: number, b: number)` … 各引数の後ろに `: 型`。
- `): number =>` … 引数カッコの閉じ `)` の後ろの `: number` が **戻り値の型**。

本アプリの実例: `formatTyoku = (value: string | number): string => { ... }` (§19)。

「`value` は文字列または数値、戻り値は文字列」と宣言しています。`getDayOfWeek = (dateStr: string): string => { ... }` も同様です。

14.5 union型（複数の型のどれか）

|（縦棒）で「AまたはB」という型を作れます。

```
let value: string | number; // 文字列 または 数値（※説明用の簡易例）
value = "abc"; // ✅ OK
value = 123; // ✅ OK
value = true; // ❌ エラー（boolean は許していない）
```

用語: union型（ユニオン型／合併型： `A | B` の形で「AかBのどちらか」を表す型） 「`null` かもしれない」を表す `string | null` が特に頻出です。

本アプリの実例:

- `formatTyoku` の引数 `value: string | number` … 文字列でも数値でも受け取れる。
- `KosuList.tsx`: `useState<string | null>(null)` … 「文字列、または `null`」。エラーメッセージは「無いとき(`null`)」と「あるとき(文字列)」があるため。

14.6 interface — オブジェクトの「設計図」

interface（インターフェース）は、**オブジェクトがどんなキーと型を持つか** を定義する「設計図」です。本アプリで最重要のTS機能です。

▼ **このコードがやること**：「従業員オブジェクトは、番号(数値)・名前(文字列)・管理者か(真偽値)を持つ」という設計図を定義し、それに沿ったオブジェクトを作ります。

```
interface Employee {           // Employee という設計図を定義（※説明用の簡易例）
  employeeNo: number;          // 番号は数値
  name: string;                // 名前は文字列
  isAdmin: boolean;            // 管理者かは真偽値
}

const e: Employee = {          // この設計図に従ったオブジェクトを作る
  employeeNo: 123,
  name: "山田",
  isAdmin: false,
};

const bad: Employee = {        // ✖ エラー！ name が抜けている & 型が違う
  employeeNo: "123",           // employeeNo は number のはずが文字列
};
```

分解すると：

- `interface 名前 { キー: 型; ... }` … オブジェクトの形を定義。各行の終わりは `;`。
- これを `: Employee` のように型注釈で使うと、「このオブジェクトは必ずこの形」と保証される。
- 形が違う（キーが足りない・型が違う）と、**書いた瞬間にエラー** で気づける。

なぜ interface が嬉しいの？ サーバーから来るデータの「形」を設計図にしておけば、`item.name` と書くときにエディタが候補を出してくれ（補完）、`item.nmae`（タイプミス）はエラーになります。「データの形を忘れた・打ち間違えた」という事故を激減させます。

★**本アプリの中心**★ `KosuList.tsx` の冒頭にある `interface Kosu { ... }` (§19で全行解説) が、まさにこれです。サーバーから来る1件の工数データの形を定義しています。

14.7 型エイリアス (type)

`type` でも型に名前を付けられます。 `interface` と似ていますが、union型などにも名前を付けられる点の違いがあります。

```
type ID = string | number; // 「文字列か数値」に ID という別名を付ける (※説明用の簡易例)
type Status = "success" | "error" | "loading"; // 決まった文字列だけを許す型
```

interface と type の使い分け (ざっくり) : 「オブジェクトの形」なら `interface`、「union や別名」なら `type`、と覚えておけば実務では十分です。本アプリはオブジェクトの形に `interface` (`Kosu`) を使っています。

15. 非同期処理 — Promise と async / await

この章で **最も重要かつ、最初は難しい** テーマです。本アプリの「サーバーと通信する処理」の根幹なので、たとえ話でじっくり理解しましょう。

15.1 「非同期」とは何か

用語: 同期 (どうき / synchronous) と非同期 (ひどうき / asynchronous)

- **同期** … 1つの処理が終わるまで、次に進まず **待つ** こと。
- **非同期** … 時間のかかる処理を **待っている間に、他のことを進める** こと。

たとえ話: レストランの注文 あなたが料理を注文しました。料理ができるまで **10分** かかりません。

- **同期だと:** 注文した瞬間からあなたは硬直し、料理が来るまで10分間、水も飲まず会話もせず固まる。 **店全体が止まる** (他のお客さんの注文も取れない)。
- **非同期だと:** 注文票を厨房に渡したら、あなたは席で会話したり水を飲んだりできる。料理ができたら店員が運んでくる。 **待っている間も店は回り続ける**。

Webアプリで「サーバーにデータを問い合わせる」のは、まさに「料理を注文する」ようなもの。ネット越しの通信には時間がかかります。その間、**画面が固まらないように**するのが非同期処理です。もし同期で書いたら、サーバーの返事を待つ数秒間、画面が一切操作できなくなってしまうです。

15.2 Promise（約束手形）

非同期処理の結果を表すのが **Promise（プロミス）** です。

用語: Promise（プロミス／約束：今すぐには結果が出ないが「後で必ず結果を渡す」という約束手形のようなもの） レストランの「注文票・呼び出しベル」に相当します。Promiseは3つの状態を持ちます：

- **pending（ペンディング）** … 処理中（料理を作っている最中）。
- **fulfilled（フルフィルド）** … 成功（料理ができた）。
- **rejected（リJECTED）** … 失敗（材料切れで作れなかった）。

通信ライブラリ（axios、§17）は、`api.post(...)` を呼ぶと **すぐにPromise（注文票）を返します**。実際のデータはまだ来ていません。

15.3 async / await（約束の結果を待つ）

Promiseを **読みやすく扱う** ための仕組みが `async` と `await` です。

▼ **このコードがやること:** サーバーにログイン情報を送り、その「返事が来るまで待ってから」次の処理に進みます。これは `Login.tsx` の処理の骨格そのものです。

```
const handleSubmit = async (event) => { // ① async: この関数は非同期処理を含む（※Promise）
  const response = await api.post("/api/login/", { ... }); // ② await: 返事が来るまで待つ
  const data = response.data; // ③ 返事が来てから、この行が実行される
  console.log(data);
};
```

分解すると：

- `async` … 関数の前に付ける。「この関数の中で `await` を使う（非同期処理を含む）」という宣言。
- `await` … Promise（注文票）の前に付ける。「この **約束の結果が来るまで待ってから** 次に進む」という意味。
- `await` を付けると、`api.post` が返したPromiseが解決（料理が完成）するまで、その行で **静かに待機** します。**待っている間、画面の他の部分は固まりません**（非同期だから）。結果が出たら、続きの行へ進みます。

async と await はセット: `await` は `async` を付けた関数の中でしか使えません。「`async` で『これは注文係です』と宣言し、その中で `await` で『この注文の完成を待つ』」という関係です。

なぜ便利? `await` がないと、Promiseの結果は `.then(...)` というコールバック（後続処理）で受け取る必要があり、処理が入れ子になって読みにくくなります。`await` を使うと、非同期なのに **まるで上から下へ普通に読める** コードになります。これが `async / await` の最大の利点です。

15.4 try / catch（失敗に備える）

通信は **失敗することがあります**（サーバー停止、ネット切断、パスワード違いなど）。失敗に備えるのが `try / catch` です。

▼ **このコードがやること:** 通信を試み（try）、もし失敗したら（catch）エラー処理をします。

```
try {                                     // try: この中の処理を「試す」（※説明用の簡易例）
  const response = await api.post(...); // 成功すればそのまま下へ
  console.log("成功", response);
} catch (err) {                           // catch: try の中でエラーが起きたら、ここに飛ん
  console.log("失敗", err);               // err にエラー情報が入っている
} finally {                               // finally: 成功・失敗どちらでも最後に必ず実行（
  console.log("通信おわり");
}
```

分解すると：

- `try { ... }` … 「失敗するかもしれない処理」を入れる。
- `catch (err) { ... }` … `try` の中でエラー（例外）が起きると、即座にここへジャンプ。`err` にエラーの中身が入る。
- `finally { ... }` … 成功・失敗に **関係なく** 最後に実行（後片付け用。例：ローディング表示を消す）。

たとえば: `try` は「綱渡りに挑戦」、`catch` は「下に張った安全ネット」。落ちても（エラーが起きてても）ネットが受け止め、アプリ全体がクラッシュするのを防ぎます。

★本アプリの実例★

- `Login.tsx`: `try` でログイン通信、`catch` でエラー時にメッセージ表示（§18）。
- `KosuList.tsx`: `try` でデータ取得、`catch` でエラー処理、`finally` でローディング表示をオフ（§19）。通信処理は **必ず** `try/catch` で囲む、と覚えてください。

16. import / export — ファイルをまたぐ

実際のアプリは、コードを **たくさんのファイルに分割** します。ファイル間でコードを受け渡すのが `import` / `export` です。

16.1 export（外に渡す）と import（受け取る）

▼ **このコードがやること:** 1つのファイルで作った関数を「外に公開」し、別のファイルで「受け取って使う」流れを示します。

```
// ファイルA: math.ts（※説明用の簡易例）
export const add = (a, b) => a + b; // export: この add を外のファイルから使えるようにする
```

```
// ファイルB: 別のファイル (※説明用の簡易例)
import { add } from "./math"; // math.ts から add を取り込む
console.log(add(2, 3));       // 5
```

- `export` … 「このファイルの外に公開する」印。
- `import { 名前 } from "パス"` … 別ファイルから取り込む。`./` は「同じフォルダ」を表す相対パス。

16.2 名前付き export と デフォルト export

種類	export側	import側	特徴
名前付き	<code>export const add =</code> <code>...</code>	<code>import { add } from</code> <code>"..."</code>	<code>{ }</code> で囲む。複数可
デフォルト	<code>export default Login;</code>	<code>import Login from "..."</code>	<code>{ }</code> なし。1ファイル1つ

本アプリの実例 (§ 17～§ 19で実際に見ます) :

- `axios.ts` : `export default api;` → `Login.tsx` で `import api from "../api/axios";` (デフォルト)。
- `Login.tsx` : `export default Login;` → 別の場所で `import Login from ...` (デフォルト)。
- `KosuList.tsx` : `import React, { useState, useEffect, ... } from "react";` → React 本体 (デフォルト) と `useState` 等 (名前付き) を **同時に** 取り込む。
- `import { Link, useNavigate, useLocation } from "react-router-dom";` → 名前付きで複数取り込む。

../ と ./ の意味: import文のパスにある記号です。

- `./` … 今のファイルと **同じフォルダ**。
- `../` … **1つ上のフォルダ**。 `../api/axios` は「1つ上に行ってから api フォルダの axios」。

- "react" のように `./` や `../` が付かないものは、インストールした外部ライブラリ（`node_modules`）から取り込まれます。

17. 実コード全解説：axios.ts 全8行

ここからは本アプリの実コードです。まずは最も短い `frontend/src/api/axios.ts` を全行読みます。これは **アプリ全体の「サーバー通信の窓口」** を作る、小さくて超重要なファイルです。

用語: axios（アクシオス：ブラウザからサーバーへ通信するための人気ライブラリ）

`api.get(...)`（取得）や `api.post(...)`（送信）で、サーバーとデータをやり取りします。
§ 15で学んだ通り、これらはPromiseを返す **非同期** 処理です。

▼ **このコードがやること（先に日本語で）**：「サーバーのアドレス」「ログイン情報（クッキー）を一緒に送る設定」「セキュリティ対策（CSRFトークン）の設定」をあらかじめ仕込んだ**専用の通信オブジェクト** `api` を1つ作り、アプリ全体で使い回せるように外へ公開します。

```
import axios from "axios";                                // ① axiosライブラリを取り込む
const api = axios.create({                                  // ② 設定済みの通信オブジェクトを作る
  baseURL: process.env.REACT_APP_API_BASE_URL,            // ③ 通信先サーバーの基本アドレス
  withCredentials: true,                                   // ④ クッキー（ログイン情報）を一緒に
  xsrfCookieName: "csrftoken",                             // ⑤ CSRF対策トークンが入るクッキー名
  xsrfHeaderName: "X-CSRFToken",                          // ⑥ そのトークンを送るヘッダー名
});                                                         // （設定オブジェクトの閉じ）
export default api;                                         // ⑦ 作った api を外のファイルへ公開
```

1行ずつ詳しく：

- ① `import axios from "axios";` 外部ライブラリ `axios` を取り込みます（§ 16）。`from "axios"` に `./` が付いていないので、これはインストール済みの外部ライブラリ（`node_modules`）からの取り込みです。これで `axios` という名前が使えるようになります。
- ② `const api = axios.create({ ... });` `axios.create(...)` で、**設定をあらかじめ仕込んだ専用の通信オブジェクト** を作り、`api` という `const` の箱に入れます（§ 2）。カッコの中の `{ ... }` は **設定オブジェクト**（§ 12）で、③～⑥の各設定がキー:値のペアで並んでいます。

なぜ `axios.create` で専用版を作るの？ 毎回 `axios.get("http://サーバー/...", { 設定 })` と書くと、サーバーアドレスや共通設定を **毎回繰り返す** ことになり、ミスのもとです。最初に1回 `api` を作っておけば、各画面では `api.get("/api/...")` と **短く書け、設定も統一** されます。「自分専用のカスタマイズした道具を1本用意する」イメージです。

③ `baseUrl: process.env.REACT_APP_API_BASE_URL`, 通信先サーバーの **基本アドレス** を指定します。

- `process.env.REACT_APP_API_BASE_URL` ... `.env` ファイル (03 章の環境設定で触れたファイル) に書かれた **環境変数** の値。例えば本番なら `https://本番サーバー`、開発中なら `http://localhost:8000` のように、**環境によって切り替えられる** 値です。

用語: 環境変数 (かんきょうへんすう / environment variable : プログラム本体に書かず外部に置く設定値) 「開発用」と「本番用」でサーバーアドレスを切り替えたいときに使います。コードを書き換えずに接続先を変えられるのが利点です。 `process.env.〇〇` で読み取ります。React では名前を `REACT_APP_` で始めるルールがあります。

baseUrl の効果: これを設定しておくと、各画面で `api.post("/api/login/")` と書くだけで、自動的に `baseUrl + "/api/login/"` (例: `http://localhost:8000/api/login/`) という完全なアドレスに通信してくれます。 `Login.tsx` の `api.post("/api/login/", ...)` が短く書けるのはこのおかげです。

④ `withCredentials: true`, 「通信のときに **クッキーを一緒に送る**」という設定です。

用語: クッキー (cookie : ブラウザがサーバーから預かって保存する小さなデータ) 本アプリでは「ログイン済みかどうか」を示す **セッション情報** がここに入ります。 `withCredentials: true` にしないと、せっかくログインしてもクッキーが送られず「誰だかわからない」扱いになります。CLAUDE.mdにある「セッション認証」を成立させるための必須設定です。

⑤ `xsrftokenName: "csrftoken"`, ⑥ `xsrftokenName: "X-CSRFToken"`, **CSRF (シーサーフ) 対策** の設定です。

用語: CSRF（クロスサイト・リクエスト・フォージェリ：ログイン中のあなたになりすまして勝手にリクエストを送る攻撃） これを防ぐため、サーバーは「合言葉（CSRFトークン）」を発行し、データ送信時にその合言葉を一緒に送らせます。

- ⑤ `xsrCookieName: "csrftoken"` … サーバーが合言葉を入れてくる **クッキーの名前**。
- ⑥ `xsrHeaderName: "X-CSRFToken"` … その合言葉を **送り返すときのヘッダー名**。この2行で「クッキーから合言葉を取り出し、送信時に決まったヘッダーで送り返す」処理を `axios` が自動でやってくれます。Django（バックエンド）の標準的なCSRF対策に合わせた名前です。

用語: ヘッダー（header：通信データに付ける「送り状・付箋」のような追加情報） 本体（データ）とは別に、「これは誰からか」「合言葉はこれ」などのメタ情報を運びます。

⑦ `export default api;` 作った `api` を **デフォルトexport** (§ 16.2) で外に公開します。これにより、他のファイルが `import api from "../api/axios";`（`Login.tsx` の3行目、`KosuList.tsx` の2行目）で受け取って使えます。

▼ **こう理解できれば成功:** この8行は「アプリ全体で共通の、設定済み通信オブジェクト `api` を1個作って配る」ファイル。以降のすべての画面は、この `api` を `import` して `api.get / api.post` でサーバーと通信します。**通信の入り口は、すべてこの1ファイルに集約されている** のです。

18. 実コード全解説：Login.tsx の handleSubmit

次に、ログイン画面の **処理本体** `handleSubmit` を1行ずつ読みます。「ログインボタンを押したら何が起きるか」の全貌です。`03` 章ではこの画面のJSX（見た目）を読みましたが、今回はその **動き** を読みます。

まず、関連する上部のコードから。

18.1 import と state (1~11行目)

▼ このコードがやること: 必要な部品を取り込み、「入力された従業員番号」「エラーメッセージ」という2つの状態の箱と、ページ移動の道具を用意します。

```
import React, { useState } from "react"; // ① React本体と useState を取り込み
import { Link, useNavigate } from "react-router-dom"; // ② リンクとページ移動の道具
import api from "../api/axios"; // ③ §17で作った通信オブジェクト
import axios from "axios"; // ④ エラー判定に使う axios 本体
import logo from "../img/TitleRogo.png"; // ⑤ ロゴ画像ファイル
import styles from "../styles/MainPage/Login.module.css"; // ⑥ この画面専用CSS (03章)

const Login: React.FC = () => { // ⑦ Loginコンポーネントの定義
  const [employee_no, setNumber] = useState<string>(""); // ⑧ 入力値の状態 (最初は空文字)
  const [errorMessage, setErrorMessage] = useState<string>(""); // ⑨ エラー文の状態 (最初は空文字)
  const navigate = useNavigate(); // ⑩ ページ移動関数を取得
```

- ①~⑥ … import (§16)。Reactの機能、ルーティング (画面遷移) の道具、§17の `api`、エラー判定用の `axios` 本体、画像、CSSを取り込んでいます。 `react` と `react-router-dom` は外部ライブラリ、`../` 付きは自作ファイルです。
- ⑦ `const Login: React.FC = () => { … Login` という **コンポーネント** (画面部品。05章で詳説) をアロー関数で定義。 `: React.FC` は「これはReactの関数コンポーネントという型ですよ」という型注釈 (§14) です。
- ⑧ `const [employee_no, setNumber] = useState<string>("");` … ここが重要。
 - `useState` (05章で詳説) は「画面と連動する状態の箱」を作るReactの機能。
 - `<string>` は型引数 (§14) で「この状態は文字列型」の指定。
 - `("")` は初期値「空文字」。
 - 戻り値は **配列** で、それを **分割代入** (§13.1) して、`employee_no` (今の値) と `setNumber` (値を変える関数) を受け取っています。
 - つまり「入力された従業員番号を覚えておく箱と、その箱を更新する道具」のセット。
- ⑨ … 同様に、エラーメッセージ用の状態 `errorMessage` と更新関数 `setErrorMessage`。最初は空文字 (=エラーなし)。
- ⑩ `const navigate = useNavigate();` … 「別のページへ移動する」関数 `navigate` を取得。後で `navigate("/")` のように使い、ログイン成功後にトップへ飛ばします。

18.2 handleSubmit 本体（13～31行目）★この章の核心★

これがログインボタンを押したときに走る処理です。§ 15で学んだ `async` / `await` / `try` / `catch` がフル稼働します。

▼ このコードがやること（先に日本語で）：①フォーム送信の既定動作を止め、②前のエラー表示を消し、③サーバーに従業員番号を送ってログインを試み、④成功ならトップページへ移動、⑤失敗理由を画面に表示します。通信は非同期なので `async/await`、失敗に備えて `try/catch` を使います。

```
const handleSubmit = async (event: React.FormEvent) => { // ① 非同期関数として定義
  event.preventDefault(); // ② フォームの既定送信を止める
  setErrorMessage(""); // ③ 前回のエラー表示を消す

  try { // ④ 通信を試す（失敗に備える）
    const response = await api.post("/api/login/", { employee_no: Number(employee_no) });
    const data = response.data; // ⑥ 返事の中身を取り出す
    if (data.status === "success") { // ⑦ 成功と書いてあれば
      navigate("/"); // ⑧ トップページへ移動
    } else { // ⑨ そうでなければ（業務的な失敗）
      setErrorMessage(data.message || "サーバーエラーが発生しました。"); // ⑩ サーバーの
    }
  } catch (err) { // ⑪ 通信自体が失敗したら
    if (axios.isAxiosError(err)) { // ⑫ axiosの通信エラーなら
      if (err.response?.status === 401) navigate("/login"); // ⑬ 401(未認証)ならログイン画面へ
      else setErrorMessage(err.response?.data.message); // ⑭ それ以外はサーバーの理由
    } else setErrorMessage("不明なエラーが発生しました。IT担当者に連絡してください。");
  }
};
```

1行ずつ詳しく：

① `const handleSubmit = async (event: React.FormEvent) => {` `handleSubmit` を **アロー関数** (§ 9) で定義。先頭の `async` (§ 15.3) で「中で `await` を使う非同期関数」だと宣言。引数 `event` は「フォーム送信イベント（どんな出来事が起きたかの情報）」で、型は `React.FormEvent` (§ 14の型注釈)。

用語: イベント（event：ユーザー操作で起きる出来事） 「ボタンが押された」「フォームが送信された」「キーが押された」など。その情報が `event` オブジェクトに入って関数に渡されま

す。

- ② `event.preventDefault();` `preventDefault()` は「**ブラウザの既定の動作を止める**」命令。

なぜ必要？ HTMLの `<form>` は、送信ボタンを押すと **既定でページ全体を再読み込み** してサーバーに飛ばうとします（昔ながらの動き）。それをこの1行で止め、代わりに自分たちの `handleSubmit`（JavaScriptでの通信）で処理します。これがないと、入力した瞬間にページがリロードされてReactの状態が消えてしまいます。**フォーム処理の定番のおまじない** です。

- ③ `setErrorMessage("");` エラーメッセージの状態を空文字に戻し、**前回の失敗時に出ていた赤いエラー表示を消します**（§18.1⑨の `setErrorMessage`）。再挑戦のたびに古いエラーが残らないようにする配慮です。

- ④ `try {` ここから「失敗するかもしれない通信処理」の開始（§15.4）。中でエラーが起きたら⑪の `catch` に飛びます。

- ⑤ `const response = await api.post("/api/login/", { employee_no: Number(employee_no) });`

★最重要行★ この1行が「サーバーにログインを依頼する」中心です。分解します：

- `api.post(...)` … §17で作った `api` を使い、サーバーに **POST通信**（データを送る通信）をする。Promise（§15.2、注文票）を返す。
- `"/api/login/"` … 送信先のアドレス。 `baseUrl` と合わさって完全なURLになる（§17③）。
- `{ employee_no: Number(employee_no) }` … 送るデータ（オブジェクト、§12）。キー `employee_no` に、入力値を数値化したものを入れる。
- `Number(employee_no)` … ★ポイント★ 入力欄から来る値は **文字列**（03 §2.2の注記、`<input type="number">` でも文字列で届く）。 `Number(...)` で **数値に変換** してから送っています。サーバーは数値を期待しているためです（型の取り違えを防ぐ）。
- `await` … この通信の **返事が来るまで待つ**（§15.3）。待っている間、画面は固まりません。
- `const response = ...` … 返ってきた結果（サーバーの応答）を `response` に受け取る。

用語: POST と GET（サーバー通信の種類）

- **GET** … データを **取得** する（読む）。 `KosuList` の一覧取得など。
- **POST** … データを **送る・登録する**（書く）。ログイン情報の送信など。ログインは「番号を送って認証してもらう」ので POST です。

⑥ `const data = response.data;` サーバーからの応答 `response` の中の `data` (実際の中身) を取り出します (§ 12.1のドット記法)。 `response` には通信のステータスなど色々入っていますが、欲しいのは中身の `data` です。この `data` には、サーバーが返した `{ status: ..., message: ... }` のようなオブジェクトが入っています。

⑦ `if (data.status === "success") {` `data` の中の `status` が文字列 `"success"` と **厳密に等しいか** (§ 4.3の `===`) を判定。サーバーが「ログイン成功」を `status: "success"` で知らせてくる約束になっているのです。

⑧ `navigate("/")`; 成功なら、 `navigate` (§ 18.1⑩) で **トップページ /** へ移動。ログイン後の画面に切り替わります。

⑨ `} else {` `status` が `"success"` でなかった場合 (=サーバーは応答したが「ログインは失敗」と言っている、業務上の失敗。例: 番号が登録されていない)。

⑩ `setErrorMessage(data.message || "サーバーエラーが発生しました。");` 失敗理由を画面に表示するため、エラーメッセージ状態を更新。

- `data.message` … サーバーが返した具体的な理由 (例: 「該当する従業員がいません」)。
- `|| "サーバーエラーが発生しました。"` … § 4.4の「ないときの保険」。もし `data.message` が空なら、代わりに「サーバーエラーが発生しました。」を表示。
- ここで `errorMessage` 状態が変わると、JSX (03 § 3⑨の `{errorMessage && ...}`) が反応して画面に赤いアラートが出ます。

⑪ `} catch (err) {` ここからは `try` の中で **例外 (通信そのものの失敗)** が起きたときの処理 (§ 15.4)。ネット切断、サーバーダウン、認証拒否 (401) などはここに来ます。 `err` にエラー情報が入ります。

⑫ `if (axios.isAxiosError(err)) {` `axios.isAxiosError(err)` は「このエラーは **axiosの通信エラーかどうか**」を判定する関数 (④でimportした `axios` 本体の機能)。通信エラーなら、 `err.response` (サーバーの応答) から詳しい情報が読めます。

⑬ `if (err.response?.status === 401) navigate("/login");` `err.response?.status` (§ 13.3のオプションルチェイニング `?.` で安全にアクセス) が **401** なら、ログイン画面へ移動。

用語: HTTPステータスコード (サーバーが返す「結果を表す番号」) **200** = 成功、**401** = 未認証 (ログインが必要)、**403** = 権限なし、**404** = 見つからない、**500** = サーバー内部エラー。**401** は「あなたが誰だか分からない (ログインして)」の意味なので、ログイン画面へ送ります。CLAUDE.mdの「401はログインヘリダイレクト」という方針が、この行で実装されて

います。ここで `?.` を使う理由は、ネットワーク断などで `err.response` 自体が存在しないことがあり、その場合に画面がクラッシュしないようにするためです。

⑭ `else setErrorMessage(err.response?.data.message);` 401以外の通信エラーなら、サーバーが返したエラーメッセージ（`err.response?.data.message`）を画面に表示します。

⑮ `} else setErrorMessage("不明なエラーが発生しました。IT担当者に連絡してください。");`
`axios.isAxiosError(err)` が `false` = `axios`以外の予想外のエラー。原因不明なので、利用者には「IT担当者に連絡」を促す汎用メッセージを表示します。

▼ こう動けば成功:

- 正しい従業員番号を入れてログイン → トップページに切り替わる (⑧)。
- 存在しない番号を入れる → 画面に赤いアラートで理由が表示される (⑩や⑭)。
- サーバーを止めた状態でログイン → 「不明なエラー…」または通信エラーのメッセージ (⑮や⑭)。

この関数の全体像（まとめ）：「①既定動作を止め→②古いエラーを消し→④⑤通信を試し→⑦⑧成功なら遷移、⑩失敗理由を表示→⑪～⑮通信エラーは種類別に対処」。通信処理の理想的なテンプレートです。本アプリの他の通信処理も、ほぼこの形を踏襲しています。

19. 実コード全解説：KosuList.tsx の型と関数とmap

最後に、工数履歴一覧画面 `KosuList.tsx` から、この章のテーマに直結する部分——`** interface Kosu` ・ `formatTyoku` ・ `getDayOfWeek` ・ データ取得 ・ `map` `**`——を1行ずつ読みます。

19.1 interface Kosu — データの設計図（10～17行目）

▼ このコードがやること: サーバーから来る「工数1件」のデータが、どんなキーとどんな型を持つかを定義する設計図です (§ 14.6)。

```
interface Kosu { // 「工数1件」の形を定義する設計図
  id: number; // 一意の番号（数値）。各行を区別する
  employee_no3: number; // 従業員番号（数値）
  name: string; // 氏名（文字列）
  work_day2: string; // 就業日（文字列。"2026-06-01" のような日付文字列）
  tyoku2: string; // 直（シフト区分。文字列。"1"～"6"）
  judgement: boolean; // 整合性（真偽値。trueならOK、falseならNG）
}
```

1行ずつ：

- `interface Kosu {` … `Kosu` という名前の設計図を定義開始（§ 14.6）。
- `id: number;` … キー `id` は数値型。各データを区別する一意の番号。後で `key={item.id}` や URL 作りに使う。
- `employee_no3: number;` … 従業員番号、数値型。
- `name: string;` … 氏名、文字列型。
- `work_day2: string;` … 就業日。日付ですが、サーバーからは `"2026-06-01"` のような **文字列** で来るので `string`。
- `tyoku2: string;` … 直（勤務シフト区分）。`"1" ～ "6"` の文字列。これを `formatTyoku` で「1直」などに変換します。
- `judgement: boolean;` … 整合性チェック結果。`true`（OK）か `false`（NG）の真偽値（§ 3.1）。
- 各行の終わりは ****セミコロン ;****。

この interface が効く場所: 下で `useState<Kosu[]>([])` と書くことで、「`data` は `Kosu` の配列だ」とTypeScriptに伝わります。すると `item.work_day2` と書けばエディタが補完してくれ、`item.workday2`（タイプミス）はエラーになります。設計図のおかげで、**サーバーデータを安全に扱える** のです。

用語: `Kosu[]`（配列型：型名の後ろに `[]` を付けて「その型の配列」を表す）`Kosu[]` は「`Kosu` オブジェクトがたくさん並んだ配列」。`number[]` なら数値の配列、`string[]` なら文字列の配列。§ 14の配列型です。

19.2 formatTyoku — switch で変換する関数（19～29行目）

▼ このコードがやること: 直の番号（1～6、文字列でも数値でも可）を受け取り、「1直」「常昼」などの **人間が読める文字列** に変換して返します。§ 6.3の switch、§ 14.4の関数型注釈の実例です。

```
const formatTyoku = (value: string | number): string => { // 引数はstring|number、戻り値
  switch (Number(value)) { // value を数値化して分岐
    case 1: return "1直"; // 1 なら "1直" を返す
    case 2: return "2直"; // 2 なら "2直"
    case 3: return "3直"; // 3 なら "3直"
    case 4: return "常昼"; // 4 なら "常昼"
    case 5: return "連1直"; // 5 なら "連1直"
    case 6: return "連2直"; // 6 なら "連2直"
    default: return ""; // どれでもなければ空文字
  }
};
```

1行ずつ：

- `const formatTyoku = (value: string | number): string => {`
 - アロー関数（§ 9）。引数 `value` の型は `string | number`（union型、§ 14.5）＝「文字列でも数値でも受け取れる」。interface の `tyoku2` は文字列ですが、念のため数値でも対応できるようにしています。
 - `: string =>` の `: string` は **戻り値の型**（§ 14.4）＝「必ず文字列を返す」。
- `switch (Number(value)) {`
 - `Number(value)` で引数を **数値に変換** してから分岐（§ 18⑤と同じ `Number` の活用）。`tyoku2` は `"1"` のような文字列で来るので、数値にしてから `case 1` などと照合します。
- `case 1: return "1直";` … 値が1なら `"1直"` を返して関数終了。
 - ⚠ `switch` では本来、各 `case` の最後に `break;` が必要で、書かないと次の case に **流れ落ちてしまう**（フォールスルー）バグになります。しかしここでは `return` を使っているため、その場で関数を抜けるので `break` は不要です。「`case` の中で `return` するなら `break` はいらない」と覚えておきましょう。
- `case 2 ~ case 6` … 同様に各番号を対応する名称へ。
- `default: return "";` … どの `case` にも当てはまらない想定外の値なら、空文字を返す（§ 6.3）。エラーで止まらず安全に処理する配慮です。

使われ方（後述 § 19.5）：表の中で `{formatTyoku(item.tyoku2)}` と呼ばれ、`tyoku2`（例：`"1"`）が「1直」と表示されます。

19.3 getDayOfWeek — 日付から曜日を出す関数（31～35行目）

▼ このコードがやること：`"2026-06-01"` のような日付文字列を受け取り、「日」「月」…などの **曜日の漢字1文字** を返します。配列（§ 10）と `Date` オブジェクトの実例です。

```
const getDayOfWeek = (dateStr: string): string => {           // 日付文字列を受け取り、曜日文
  const days = ["日", "月", "火", "水", "木", "金", "土"]; // 曜日の配列（番号0～6に対応）
  const date = new Date(dateStr);                             // 文字列から日付オブジェクトを
  return days[date.getDay()] || "";                             // 曜日番号で配列を引き、なけれ
};
```

1行ずつ：

- `const getDayOfWeek = (dateStr: string): string => {`
 - アロー関数。引数 `dateStr` は文字列（日付文字列）、戻り値も文字列（曜日）。
- `const days = ["日", "月", "火", "水", "木", "金", "土"];`
 - 曜日を表す7要素の **配列**（§ 10.1）。**順番が超重要**：`days[0]` が「日」、`days[1]` が「月」…
`days[6]` が「土」。これは次の `getDay()` の戻り値（0=日曜～6=土曜）に **ピッタリ対応** させてあります。
- `const date = new Date(dateStr);`
 - `new Date(...)` で、文字列から **日付オブジェクト** を作ります。

用語：`new Date(...)`（**デート**：日付・時刻を扱う標準部品） `new Date("2026-06-01")` のように日付文字列を渡すと、その日を表す日付オブジェクトができて、`.getDay()`（曜日）や `.getMonth()`（月）などで情報を取り出せます。`new` は「新しく作る」キーワード（05 章でも登場）。

- `return days[date.getDay()] || "";`
 - `date.getDay()` … その日付の **曜日番号** を返す（日曜=0、月曜=1、…、土曜=6）。

- `days[ ... ]` … その番号で曜日配列を引く。例えば2026-06-01が月曜なら `getDay()` は1、`days[1]` は「月」。
- `|| ""` … 「ないときの保険」 (§ 4.4)。万一 `dateStr` が不正で `getDay()` が `NaN` を返すと `days[NaN]` は `undefined` になるため、その場合は空文字を返してエラーを防ぎます。

この2つの関数の共通点: どちらも「サーバーから来た生のデータ（番号・日付文字列）を、人間が読みやすい表示に変換する」役割。こういう変換関数を **フォーマッター (formatter)** と呼びます。表示の直前に呼んで使います。

19.4 データ取得 — `fetchData` の中身 (51~84行目)

ここで § 13.2 (スプレッド) ・ § 4.4 (`||`) ・ § 6.2 (三項) ・ § 15 (`async/try/catch`) が一気に登場します。

▼ **このコードがやること:** サーバーに「工数一覧をください。ページ番号はこれ、(日付指定があれば) この日で絞り込んで」と非同期で問い合わせ、結果を画面の状態にセットします。エラーは種類別に対処し、最後に必ずローディング表示を消します。


```

const fetchData = useCallback(async (
  page: number, // ① 引数：ページ番号
  day: string, // ② 引数：検索する日付（空なら絞り込みなし）
  mode: boolean // ③ 引数：月検索ならtrue、日検索ならfalse
) => {
  setLoading(true); // ④ ローディング表示をオンに
  try { // ⑤ 通信を試す
    const response = await api.get("/api/kosu_list/", { // ⑥ 一覧をGET取得（返事を待つ）
      params: { // ⑦ サーバーへ渡す検索条件（クエリパラメータ）
        page: page, // ページ番号は必ず送る
        ...(day && { // ⑧ day に値があるときだけ、以下を展開して追加
          day: day, // 検索日
          mode: mode ? "month" : "day", // 月検索か日検索か（三項演算子）
          filter: "true", // 絞り込みフラグ
        })
      }
    });

    const results = response.data.results || []; // ⑨ 結果配列（なければ空配列）
    const pageSize = response.data.page_size || 20; // ⑩ 1ページ件数（なければ20）
    setData(results); // ⑪ 取得データを状態にセット
    setTotalPages(Math.ceil(response.data.count / pageSize)); // ⑫ 総ページ数を計算して
  } catch (err) { // ⑬ 通信失敗時
    if (axios.isAxiosError(err)) { // ⑭ axiosの通信エラーなら
      if (err.response?.status === 401) navigate("/login"); // ⑮ 401ならログインへ
      else if (err.response?.status === 403) navigate("/"); // ⑯ 403（権限なし）ならトップへ
      else setError(err.response?.data.message); // ⑰ その他はエラー文を表示
    } else {
      setError("不明なエラーが発生しました。IT担当者に連絡してください。"); // ⑱ axios以外のエラー
    }
  } finally { // ⑲ 成功・失敗どちらでも最後に
    setLoading(false); // ⑳ ローディング表示をオフに
  }
}, [navigate]);

```

ポイントを絞って解説します（`useCallback` 自体は 05 章のReactで詳説。ここでは中の処理に集中します）：

- ① `setLoading(true);` … 読み込み中フラグをオン。これで画面に「読み込み中…」表示（`Loading`）が出ます（104行目で使用）。
- ② `try {` … 通信開始。 `Login.tsx` 同様、必ず `try` で囲みます。
- ③ `const response = await api.get("/api/kosu_list/", { ... });` … `api.get` (§17の `api`) で一覧データを 取得（GET）。`await` で返事を待つ。ログインは送信なので `post` でしたが、こちらは取得なので `get` です。

- ④ `params: { ... }` … GETの **検索条件** (クエリパラメータ)。`{ page: 2 }` なら、URLが `/api/kosu_list/?page=2` のようになり、サーバーに条件が伝わります。
- ⑤ `...(day && { ... })` ★高度なテクニック★ … §13.2のスプレッドと §4.4の `&&` の合わせ技。
 - `day && { ... }` … `day` (日付) に値があれば右の `{ day, mode, filter }` オブジェクトを、空なら `""` (何も足さない値) を返す。
 - `...` で、それを `params` の中に **展開** する。
 - 結果: 「日付未入力なら `page` だけ送る / 日付入力済みなら `page + day + mode + filter` を送る」という、**条件によって送る項目を変える** 賢い組み立てになっています。
 - `mode: mode ? "month" : "day"` … 三項演算子 (§6.2)。`mode` が true (指定月ボタン) なら `"month"`、false (指定日ボタン) なら `"day"`。
- ⑥ `const results = response.data.results || [];` … 応答の中の `results` (データ配列) を取り出す。`|| []` で「なければ空配列」を保証 (§4.4)。これで後の `.map` が必ず配列に対して動き、エラーになりません。
- ⑦ `const pageSize = response.data.page_size || 20;` … 1ページあたりの件数。なければ既定20。
- ⑧ `setData(results);` … 取得した配列を状態 `data` にセット。これで画面の表が更新されます。
- ⑨ `setTotalPages(Math.ceil(response.data.count / pageSize));` … 総ページ数を計算してセット。
 - `response.data.count` … 全データ件数 (例: 45件)。
 - `count / pageSize` … $45 / 20 = 2.25$ 。
 - `Math.ceil(...)` … **切り上げ**。2.25 → 3。45件を20件ずつ表示すると3ページ必要、という計算。

用語: `Math.ceil(...)` (マスシール: 数値を切り上げる関数) `Math.ceil(2.25)` は 3。「半端があれば次のページが要る」ので切り上げが正解です。`Math` は数学関連の機能をまとめた標準オブジェクト (`Math.floor` は切り捨て、`Math.round` は四捨五入)。

- ⑩~⑮ `catch` … `Login.tsx` とほぼ同じエラー処理。違いは⑬で **403 (権限なし)** のときトップへ送る点。「ログインはしているが、この画面を見る権限がない」場合の対処です。
- ⑯⑰ `finally { setLoading(false); }` … 成功でも失敗でも **必ず** ローディング表示を消す (§15.4)。`finally` がなければ、エラー時に「読み込み中…」が出っぱなしになってしまいます。

try/catch/finally の役割分担（再確認）： **try** = 挑戦、 **catch** = 失敗時の対処（種別）、 **finally** = 成否に関わらない後片付け（ローディング解除）。この3点セットが通信処理の完成形です。

19.5 描画部分の map と三項演算子（150～200行目）

最後に、取得したデータを表に展開する部分です。§ 11の **map**、§ 6.2の三項演算子、§ 5のテンプレートリテラルがすべて登場します。

▼ **このコードがやること**： データが0件なら「No data found.」を出し、あれば各データを表の1行（**<tr>**）に変換して並べます。各行では日付に曜日を添え、直を名称化し、整合性をOK/NGで色分け表示します。

```
{data.length === 0 ? ( // ① データ件数が0なら...
  <p>No data found.</p> // 「データなし」を表示
) : ( // ② そうでなければ...
  <TableContainer ...>
    <table>
      <thead> ... </thead>
      <tbody>
        {data.map((item) => ( // ③ data配列の各要素 item を、表の行に変換
          <tr key={item.id}> // ④ 1行。key には一意の id を指定
            <td>{item.work_day2} ({getDayOfWeek(item.work_day2)})</td> // ⑤ 就業日 + 曜日
            <td>{formatTyoku(item.tyoku2)}</td> // ⑥ 直を名称化
            <td className={item.judgement ? styles["status-ok"] : styles["status-ng"]} >
              {item.judgement ? "OK" : "NG"} // ⑦ 整合性の表示
            </td>
            <td>
              <Link to={` /kosu-update/${item.id}`} className={styles["a-collar"]} >編集</Link>
            </td>
            <td>
              <Link to={` /kosu-delete/${item.id}`} className={styles["a-collar"]} >削除</Link>
            </td>
          </tr>
        )]}
      </tbody>
    </table>
    <Pagination ... />
  </TableContainer>
)}
```

ポイント解説：

- ① `{data.length === 0 ? (...) : (...)}` … `data.length` (配列の件数、§ 10.1) が0かを `===` で判定し、**三項演算子** (§ 6.2) で「0件なら左、あれば右」を出し分け。これは「条件によって表示を切り替える」Reactの定番 (§ 05 章で詳説)。
- ③ `{data.map((item) => (...))}` ★この章の集大成★ … `data` (`Kosu[]` 配列) の **各要素 item を、1つの `<tr>` (表の行) に変換** (§ 11.1のmap)。「データが10件あれば10個の `<tr>` ができ、表に10行表示される」。Reactでリストを表示する最重要パターンです。
- ④ `<tr key={item.id}>` … 各行に `key` (識別用の印) として一意な `item.id` を付与。

なぜ key が必要? Reactは、データが増減したとき「どの行がどれか」を `key` で見分けて効率よく再描画します。`map` で要素を並べるときは **必ず一意な key** を付けるのがルールです (§ 05 章)。`interface Kosu` で `id` を持たせていたのは、まさにこのためでもあります。

- ⑤ `<td>{item.work_day2} ({getDayOfWeek(item.work_day2)})</td>` … 就業日 `item.work_day2` (例: `"2026-06-01"`) を表示し、その横に `getDayOfWeek` (§ 19.3) で求めた曜日をカッコ付きで添える (例: `2026-06-01 (月)`)。 `item.work_day2` は § 12のドット記法、`(...)` のカッコは普通の文字です。
- ⑥ `<td>{formatTyoku(item.tyoku2)}</td>` … 直の番号 `item.tyoku2` (例: `"1"`) を `formatTyoku` (§ 19.2) で「1直」に変換して表示。
- ⑦ `className={item.judgement ? styles["status-ok"] : styles["status-ng"]}` … 三項演算子で **CSSクラスを出し分け**。`judgement` が true なら `status-ok` (緑など)、false なら `status-ng` (赤など) のスタイルを当てる。`styles[...]` はCSS Modules (§ 03 § 13)。
- ⑧ `{item.judgement ? "OK" : "NG"}` … 真偽値 `judgement` を、人間向けに「OK」「NG」の文字で表示 (三項演算子、§ 6.2)。
- ⑨⑩ `to={` /kosu-update/${item.id}`}` … **テンプレートリテラル** (§ 5) で、`item.id` (例: 5) を埋め込んで `/kosu-update/5` というリンク先URLを動的に作る。各行の「編集」「削除」が、その行のデータのIDに対応したページへ飛ぶようになります。

▼ **こう表示されれば成功:** 一覧画面に、就業日 (曜日付き)・直 (1直など)・整合性 (OK 緑/NG赤)・編集/削除リンクが並んだ表が出る。データが無ければ「No data found.」。この表示の裏側で、本章で学んだ **map・三項・テンプレートリテラル・interface・フォーマッター関数** がすべて協働しています。

19.6 この章の文法が実コードでどう使われたか（総まとめ表）

文法（学んだ節）	KosuList.tsx での実例
interface (§ 14.6)	<code>interface Kosu { id: number; ... }</code>
配列型 (§ 14)	<code>useState<Kosu[]>([])</code>
union型 (§ 14.5)	<code>formatTyoku(value: string number)</code>
関数の型注釈 (§ 14.4)	<code>(dateStr: string): string =></code>
アロー関数 (§ 9)	<code>const formatTyoku = (value) => {...}</code>
switch (§ 6.3)	<code>formatTyoku</code> の <code>switch(Number(value))</code>
配列とインデックス (§ 10)	<code>days[date.getDay()]</code>
async/await (§ 15)	<code>await api.get(...)</code>
try/catch/finally (§ 15.4)	<code>fetchData</code> のエラー処理
オプショナルチェイニング (§ 13.3)	<code>err.response?.status</code>
スプレッド+&& (§ 13.2/ § 4.4)	<code>...(day && { ... })</code>
<code> </code> 既定値 (§ 4.4)	<code>response.data.results []</code>
三項演算子 (§ 6.2)	<code>item.judgement ? "OK" : "NG"</code>
map (§ 11.1)	<code>data.map((item) => <tr>...)</code>
テンプレートリテラル (§ 5)	<code>`/kosu-update/\${item.id}`</code>
<code>===</code> (§ 4.3)	<code>data.length === 0</code>

20. トラブルシューティング

症状	原因	対処
<code>Cannot read properties of undefined</code> で画面が真っ白	<code>null</code> / <code>undefined</code> のプロパティを <code>.</code> で辿った	オプショナルチェイニング <code>?.</code> を使う (§ 13.3)。Consoleで何が <code>undefined</code> か確認

比較がうまくいかない	<code>=</code> （代入）と <code>===</code> （比較）の取り違い	比較は必ず <code>===</code> 。 <code>if (x = 5)</code> は誤り、 <code>if (x === 5)</code> が正しい (§ 4.3)
<code>"5" === 5</code> が false で混乱	文字列と数値は別物	比較前に <code>Number(...)</code> や <code>String(...)</code> で型を揃える (§ 18⑤)
計算結果が <code>NaN</code>	文字列が数値計算に混入	<code>Number()</code> で数値化 (§ 14.1・§ 18⑤)
<code>await</code> を付けたのにエラー	関数に <code>async</code> が付いていない	<code>await</code> は <code>async</code> 関数の中だけ (§ 15.3)
通信成功なのにデータが空	<code>response</code> と <code>response.data</code> の取り違い	中身は <code>response.data</code> の中 (§ 18⑥)。Consoleで <code>response</code> を確認
<code>map</code> でエラー	対象が配列でない (undefined等)	<code> []</code> で空配列を保証してから <code>map</code> (§ 4.4・§ 19⑥)
<code>map</code> で「key」警告が出る	<code>key</code> を付けていない	<code><tr key={item.id}></code> のように一意な key を付ける (§ 19④)
「予期せぬ動作」が <code>switch</code> で起きる	<code>break</code> / <code>return</code> 忘れて次の case に流れた	各 case を <code>return</code> か <code>break</code> で締める (§ 19.2)
エラーが <code>catch</code> されず画面が固まる	通信を <code>try/catch</code> で囲っていない	通信は必ず <code>try/catch</code> (§ 15.4)
ローディングが消えない	エラー時に解除し忘れ	<code>finally</code> で <code>setLoading(false)</code> (§ 19⑩)
import でファイルが見つからない	パスの <code>./</code> <code>../</code> 間違い	<code>./</code> = 同フォルダ、 <code>../</code> = 1つ上 (§ 16.2)

21. 演習問題

手を動かすと定着します。**ローカル環境**（壊しても安全）で試してください。

- 型の感触**：`KosuList.tsx` の `interface Kosu` の `name: string;` を一時的に `name: number;` に変えると、どこかにエラー（赤線）が出ることを確認。終わったら元に戻す。「型が形を守る」ことを体感する。
- 三項演算子**：`{item.judgement ? "OK" : "NG"}` を `{item.judgement ? "合格" : "不合格"}` に変え、表の整合性列の表示が変わることを確認。元に戻す。

3. **switch を読む** : `formatTyoku` に `case 7: return "夜勤";` を追加し (`default` の前に)、もしデータに直7があればどう表示されるか考える。実際のデータには無いので表示は変わらないことも確認。
4. **map の理解** : `data.map((item) => ...)` の中で `console.log(item)` してみて、開発者ツールのConsole (03 § 14.3) に1件ずつのオブジェクトが出ることを確認。 `item` が `Kosu` オブジェクトだと体感する。
5. **テンプレートリテラル** : ``/kosu-update/${item.id}`` の `item.id` を `item.work_day2` に変えると、編集リンク先がどう変わるか (URLに日付が入る) を確認。元に戻す。
6. **async/await の流れ** : `Login.tsx` の `handleSubmit` 内、`await api.post(...)` の直後に `console.log("通信完了")`、`try` の直前に `console.log("通信開始")` を入れ、ログイン時にConsoleにこの順で出ることを確認。「`await` で待ってから次へ進む」を体感する。
7. **?. の効果** : `err.response?.status` を一時的に `err.response.status` (? を消す) に変え、サーバーを止めた状態でログインを試すとどうなるか観察 (エラーで止まる可能性)。確認後 `?.` に戻す。

22. この章のまとめ

- **JavaScript** は画面に「動き」を与える言語。**TypeScript** はそれに **型** という安全装置を足したものの。本アプリは `.ts` / `.tsx` (TypeScript)。
- **変数** は `const` (変えない・優先) と `let` (変える)。**型** は `string` / `number` / `boolean` / `null` / `undefined`。
- ****比較は必ず === **** (型まで厳密)。`=` は代入。`&&` / `||` / `!` で条件を組み、`||` は「ないときの保険」にも使う。
- **テンプレートリテラル** ``...${値}...`` で文字列に値を埋め込む。**条件分岐** は `if` / `else` ・ **三項演算子** `条件 ? A : B` ・ `switch`。
- **関数** は `function` か **アロー関数** (引数) `=> {...}` (Reactの主流)。
- **配列** は0番から。**** map **** (全要素を変換) と **** filter **** (条件で絞る) が必殺技。Reactのリスト表示は `map`。
- **オブジェクト** は名前付きの値の集まり (`obj.key`)。**分割代入** `const {a} = obj`、**スプレッド** `{...obj}`、**オプションナルチェイニング** `obj?.key` (空でも落ちない)。
- **TypeScriptの型** : 型注釈 `: 型`、型推論、**** interface (オブジェクトの設計図)**、型エイリアス、配列型 `T[]`、**union型** `A | B`。
- **非同期処理** : `Promise` (約束手形) を `async` / `await` で「待ってから次へ」読みやすく書く。失敗に備え **** try / catch / finally ****。通信処理の鉄板形。

- **import/export** でファイルをまたぐ。 `export default` と名前付き、 `./` / `../` /外部ライブラリの違い。
- 実コードでは、** `axios.ts` (通信の窓口を1つ作って配る)、 `Login.tsx` の `handleSubmit` (既定動作を止め→通信→成功で遷移・失敗で表示)、 `KosuList.tsx` ** (interfaceでデータの形を定義し、フォーマッター関数とmapで表に描く) を1行ずつ読み切った。

これでJavaScript／TypeScriptの基礎が、本物のコードと結びつきました。次は、これらを使って **画面を組み立てる** 主役、Reactへ進みます。 `useState` や `map` がどう画面に効くのか、いよいよ全貌が見えます。

次は、「[05_React_フロントエンド.md](#)」へ進みます。